FlowForge AI — Interview Presentation & Live Demo Playbook

Enterprise-Grade Visual AI Workflow Orchestration & Distributed State Engine

---

1. The 30-Second Elevator Pitch (The Hook)

> ***"Most AI tools are fragile script wrappers that crash in production when external LLMs lag or APIs timeout. I engineered FlowForge AI—an enterprise-grade visual workflow orchestration engine with a PostgreSQL-backed state machine, distributed Celery/Redis background execution, semantic RAG policy grounding, and Human-in-the-Loop (HITL) approval gates. It guarantees that high-stakes AI pipelines never execute critical decisions without cryptographic auditability, schema validation, and authorized human oversight."***

2. Enterprise Architectural Pillars

Pillar	Implementation Details & Engineering Defense
PostgreSQL Source of Truth	Workflows, step run states, execution timelines, and human approval requests are durably managed in PostgreSQL with ACID guarantees. Redis is strictly a transient message broker.
Acyclic Graph Validation	Applies Kahn's Algorithm (Topological Sort) to validate DAGs, detect cycles, find orphan/unreachable nodes, and enforce depth limits prior to publishing.
Immutable Version Snapshots	Published workflow versions are frozen with a SHA-256 checksum snapshot so running workflows are insulated against breaking schema edits.
Worker Lease Watchdog	Celery tasks acquire TTL leases in PostgreSQL. If a container or worker terminates mid-run, orphaned tasks are automatically recovered without corrupting state.
Strict AI Guardrails	All AI outputs are parsed through Pydantic V2 schemas with extra fields forbidden. Raw LLM responses never directly execute code or bypass permission checks.

3. Five-Minute Live Technical Demo Playbook

Follow this step-by-step script during technical screenings or portfolio presentations:

Step 1: Dashboard & Multi-Tenancy (0:00 - 0:45)

- Open http://localhost:5173 and showcase the modern dark-mode application shell.
- Demonstrate the organization switcher: show multi-tenant workspace isolation partitioning workflows, executions, and settings.

Step 2: Knowledge Base Grounding (0:45 - 1:30)

- Navigate to /knowledge. Showcase the seeded collections: **support_policies** and **it_runbooks**.

- Point out Document POL-BILL-001 (Refund & Billing Policy) with the explicit $100 manager approval ceiling.
- Explain: *'Workflows query this store via vector/semantic search to ground AI decisions in verified corporate policies.'*

Step 3: Visual Workflow Builder Canvas (1:30 - 2:30)

- Open /workflows and launch the **Automated Customer Support & SLA Triage** workflow.
- Showcase the React Flow DAG canvas: Trigger → AI Intent Classifier → Knowledge Retrieval → Structured Extractor → Output.
- Click on a node to show the slide-out **Node Inspector**, explaining how each node validates parameters in real-time.

Step 4: Autonomous Execution vs. Human-in-the-Loop (2:30 - 4:00)

- **Workflow A (Autonomous):** Show the completed execution for damaged goods (< $100). Highlights zero-human triage in < 3s.
- **Workflow B (Human-in-the-Loop):** Trigger or open a run with a $450 claim on the **High-Value Refund Approval Pipeline**.
- Show how the conditional node routes > $100 claims into the **Finance Manager Approval Gate**, pausing the execution safely.
- Navigate to ***/*approvals**. Show the live pending request card with full claim metadata, and click **Approve**.
- Return to Executions to show the state automatically transition to COMPLETED with an audited disbursement receipt.

Step 5: Operational Monitoring & Audit Trail (4:00 - 5:00)

- Open the **Executions Timeline**. Show step-by-step latency, token cost accounting, and masked PII (e.g. sk-••••••••).
- Conclude: *'FlowForge AI provides production dependability: deterministic DAGs, ACID state management, and real governance.'*

4. Core Workflows Comparison Architecture

Feature / Dimension	Workflow 1: Autonomous Triage	Workflow 2: High-Value Approval
Primary Objective	Instant customer support triage & response	Financial refund disbursement governance
Human Intervention	**Zero (Fully automated)**	**Mandatory for claims > $100**
Decision Node	AI Intent Classifier (categories: billing/tech)	Condition Node (> $100 threshold router)
Grounded Policies	Retrieval node querying *support_policies*	Bound by Policy POL-BILL-001 ($100 cap)
Failure Mode Handling	Auto-retries on external inference timeouts	24-hour timeout escalation if reviewer is offline

5. High-Impact Technical Interview Questions & Model Answers

Q1: Why build an orchestration engine from scratch instead of using LangChain or Flowise?

"Frameworks like LangChain or CrewAI are convenient for prototypes but lack enterprise durability. If a worker crashes mid-chain, execution state is lost. FlowForge AI models workflows as formal distributed state machines committed directly to PostgreSQL. Every step transition, attempt count, and token cost is ACID-durable, providing zero-loss crash recovery, deterministic retries, and cryptographic auditability."

Q2: How do you prevent cyclic dependencies or infinite loops on the visual canvas?

"Before a draft workflow can be published or executed, its graph passes through our validator using Kahn's Algorithm (Topological Sort). It verifies in-degrees, detects cycle participants, checks reachability from trigger nodes, and validates branching handles. Any malformed DAG is rejected with exact node and edge identifiers before reaching the database."

Q3: How do you handle Celery worker crashes during a running task?

"When a Celery worker claims an execution step, it acquires a bounded lease with a TTL in PostgreSQL. A Celery supervisor periodically queries for active steps whose heartbeats have expired. Orphaned tasks are transitioned back to QUEUED up to their configured retry ceiling, preventing unhandled job blackholes."

Q4: How do you ensure untrusted LLM outputs do not compromise system integrity?

"We treat all AI model responses as untrusted user input. We enforce strict JSON schema extraction via Pydantic V2 models with extra fields forbidden. Raw LLM responses never directly invoke internal database commands or bypass security checks. Furthermore, all sensitive keys and PII are masked in audit timeline outputs."

Q5: How does the system achieve tenant isolation in a multi-tenant setup?

"Workflows, execution histories, and reference documents include an organization_id foreign key. Every API route verifies tenant boundaries using FastAPI dependency injection (get_current_tenant). Database queries are scoped at the ORM layer, preventing cross-tenant data leakage while allowing shared global blueprint templates."